

MACHINE LEARNING ENGINEERING

FASE 4 | AULA 05 -

EXTRAÇÃO DE ENTIDADES

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	21
REFERÊNCIAS	22

O QUE VEM POR AÍ?

Imagine um mundo em que máquinas podem facilmente vasculhar vastas quantidades de texto, identificando nomes, lugares e organizações, transformando o caos em um conhecimento estruturado. Esta é a magia do Reconhecimento de Entidades Nomeadas (NER).

Essa é a base de sistemas inteligentes que entendem nossa linguagem, possibilitando desde aplicações como assistentes virtuais até motores de busca inteligentes, transformando o modo como interagimos com a tecnologia. Vamos mergulhar no fascinante mundo do NER, em que a linguagem encontra a inteligência das máquinas.

HANDS ON

Muitas vezes precisamos extrair do texto alguma informação específica, seja para responder alguma pergunta sobre os dados, estruturar o banco de dados ou para procurar dados que enriqueçam os que já temos.

Assim, a tarefa de Reconhecimento de Entidades Nomeadas (NER) visa identificar, dentro do texto, entidades como pessoas, organizações ou locais e outros.

EXEMPLO

SAIBA MAIS

Modelos de Reconhecimento de Entidades Nomeadas visam extrair do texto informações categorizadas como:

- **Pessoa (PER):** nome de pessoas.
- **Organização (ORG):** empresas e instituições.
- **Localização (LOC):** cidades, países, regiões geográficas.
- **Miscellaneous (MISC):** outros, o que não for das categorias acima, mas ainda assim for importante, como evento ou produtos.

Da mesma forma que na análise de sentimento, os modelos de reconhecimento de entidade nomeada também são de classificação, em que cada token possui uma tag e a tarefa do modelo é prever aquela tag.

O exemplo a seguir mostra como seria taggear utilizando o formato Inside-Outside-Beginning (IOB) para ser usado no treinamento de um modelo NER:

- **B- (Beginning):** marca o início de uma entidade.
- **I- (Inside):** marca o token de dentro da entidade, contínuo ao anterior.
- **O- (Outside):** marca um token que não faz parte de nenhuma entidade nomeada.

Exemplo:

Para a sentença "Barack Obama nasceu em Honolulu":

- **Barack** -> B-PER (Início da entidade de Pessoa)
- **Obama** -> I-PER (Dentro da entidade de Pessoa)
- **nasceu** -> O (Fora de qualquer entidade)
- **em** -> O
- **Honolulu** -> B-LOC (Início da entidade de Local)

Nesse formato, o modelo tem todas as informações para distinguir de forma clara quando uma entidade nomeada começa e quando termina.

Algumas bibliotecas possuem modelos pré-treinados nessa tarefa que você pode usar: a NLTK possui um modelo primariamente treinado em inglês; já a SpaCy possui o modelo 'pt_core_news_sm' que é treinado na língua portuguesa.

NLTK

```
pip install nltk
import nltk
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
nltk.download('maxent_ne_chunker')
nltk.download('words')
from nltk import pos_tag, ne_chunk
from nltk.tokenize import word_tokenize
texto = "Barack Obama nasceu em Honolulu, Havaí. Ele foi o
44° presidente dos Estados Unidos."
tokens = word_tokenize(texto)
taggeado = pos_tag(tokens)
entidades = ne_chunk(taggeado)
print(entidades)
```

Saída:

```
(S
  (PERSON Barack/NNP)
  (PERSON Obama/NNP)
  nasceu/MD
  em/VB
  (GPE Honolulu/NNP)
  ,/,
  (PERSON Havaí/NNP)
  ./.)
  (PERSON Ele/NNP)
  foi/VBD
  o/JJ
  44°/CD
  presidente/NN
  dos/NNS
  (PERSON Estados/NNP Unidos/NNP)
  ./.)
```

Mesmo com o modelo do NLTK tendo sido pré-treinado na língua inglesa, conseguimos transferir o conhecimento e usá-lo na língua portuguesa, ainda que os resultados possam não ser tão bons quanto treinarmos na língua específica. Assim, podemos ver que:

- A entidade Barack Obama como pessoa está correta, classificado como PERSON.
- A entidade Honolulu foi classificada corretamente, GPE é o mesmo que local.

As demais entidades foram classificadas erroneamente, mas este já é um começo.

E se tentássemos em inglês? Mudando o texto para "Barack Obama was born in Honolulu, Hawaii. He was the 44° president of the United States.", temos o seguinte resultado:

```
(S
  (PERSON Barack/NNP)
  (PERSON Obama/NNP)
  was/VBD
  born/VBN
  in/IN
  (GPE Honolulu/NNP)
  ,/,
  (PERSON Hawaii/NNP)
  ./
  He/PRP
  was/VBD
  the/DT
  44°/CD
  president/NN
  of/IN
  the/DT
  (GPE United/NNP States/NNPS)
  ./.)
```

Ainda assim há um pequeno erro em "Hawaii", mas já ficou muito melhor, não é mesmo?

Mesmo havendo um erro maior utilizando um modelo pré-treinado em uma outra língua, se ambas pertencem à mesma família, o modelo em geral já entrega uma performance boa.

Por exemplo: as línguas da família Indo-Europeia como espanhol, inglês, português, russo, alemão, francês e outros, possuem padrões que podem fazer com que um modelo pré-treinado em uma possa ser usado em outra. Já se fossemos testar

em textos japoneses, esses modelos teriam uma performance bem mais baixa, visto que são idiomas bem diferentes.

SpaCy

```
pip install spacy
python -m spacy download pt_core_news_sm
import spacy
nlp = spacy.load('pt_core_news_sm')
texto = "Barack Obama nasceu em Honolulu, Havaí. Ele foi o 44º presidente dos Estados Unidos."
doc = nlp(texto)
for ent in doc.ents:
    print(ent.text, ent.label_)
```

Saída:

```
Barack Obama PER
Honolulu LOC
Havaí LOC
Estados Unidos LOC
```

Perfeito!

BERT com fine-tuning

O modelo a seguir foi pré-treinado na tarefa de reconhecimento de entidades nomeadas usando como corpo o BERTimbau, que é pré-treinado na língua portuguesa. Dessa forma, uma cabeça de classificação de entidades foi adicionada usando artigos da Globo News. Mais informações sobre o modelo estão disponíveis na [documentação](#).

```
from transformers import BertForTokenClassification,
DistilBertTokenizerFast, pipeline
model = BertForTokenClassification.from_pretrained('monilouise/ner_pt_br')
tokenizer = DistilBertTokenizerFast.from_pretrained('neuralmind/bert-base-portuguese-cased',
    model_max_length=512,
    do_lower_case=False
)
ner = pipeline('ner', model=model, tokenizer=tokenizer, grouped_entities=True)
```



```
texto = "Barack Obama nasceu em Honolulu, Havaí. Ele foi o  
44° presidente dos Estados Unidos."  
entidades = ner(texto)  
for ent in entidades:  
    print(f"{ent['word']}: {ent['entity_group']}  
({ent['score']:.2f})")
```

Saída:

```
Bara: PESSOA (0.95)  
##ck: PESSOA (0.54)  
Obama: PESSOA (0.95)  
Hon: LOC (0.99)  
##olulu: LOC (0.60)  
Havaí: LOC (0.94)  
Estados: LOC (0.78)
```

Tudo certinho! Você também pode usar o modelo no servidor do Hugging Face para fazer alguns testes.

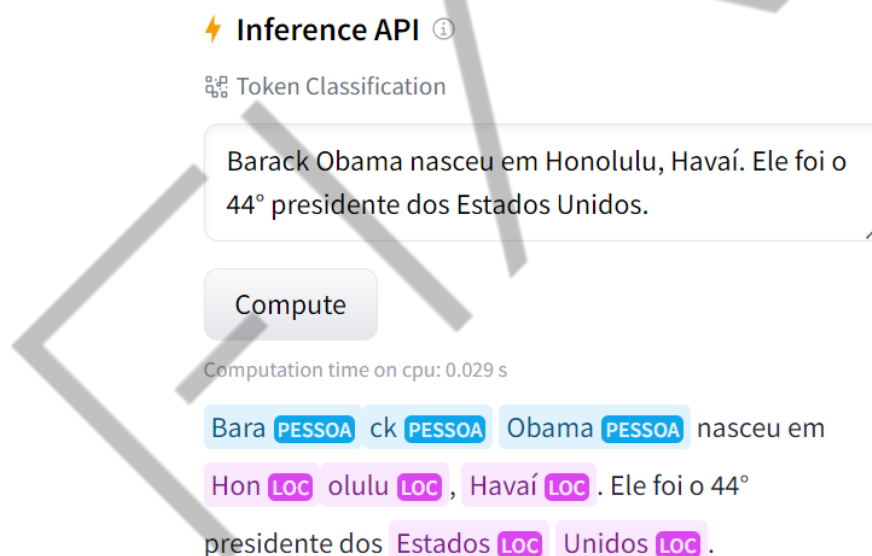


Figura 1 – Inference API
Fonte: Hugging Face (2024)

Agora que já usamos modelos prontos, vamos ver como treinar o nosso próprio?

Treinando um modelo multilinguagem

Prontos(as) para treinar um modelo multilinguagem para Reconhecimento de Entidade Nomeada (NER)? Para isso, vamos usar um pedaço do dataset “XTREME” (Cross-lingual TRansfer Evaluation of Multilingual Encoders), chamado de WikiANN ou PAN-X. Esse dataset possui muitas configurações e idiomas.

```
from datasets import get_dataset_config_names
xtreme_subsets = get_dataset_config_names("xtreme")
print(f"XTREME possui {len(xtreme_subsets)} configurações.")
panx_subsets = [s for s in xtreme_subsets if s.startswith("PAN")]
print(panx_subsets[:3])
```

Saída:

```
XTREME possui 183 configurações.
['PAN-X.af', 'PAN-X.ar', 'PAN-X.bg']
```

Os datasets possuem nomes padronizados, como podemos ver acima, com “PAN-X” e o código da língua.

Vamos pegar os seguintes idiomas: inglês, português e francês. Além disso, vamos fazer uma redução do dataset por proporções para termos uma visão mais realista. Datasets desbalanceados são muito comuns em problemas da vida real. Pense na proporção de pessoas que falam um determinado idioma: é muito mais fácil conseguir dados em inglês do que nos demais idiomas, certo?

```
from collections import defaultdict
from datasets import DatasetDict, load_dataset
import pandas as pd
langs = ['pt', 'en', 'fr']
prop = [0.13, 0.72, 0.15]
panx_ch = defaultdict(DatasetDict)
for lang, prop in zip(langs, prop):
    ds = load_dataset('xtreme', name=f'PAN-X.{lang}')
    for split in ds:
        panx_ch[lang][split] = ds[split].shuffle(seed=0).select(
            range(int(prop * ds[split].num_rows))
        )
    pd.DataFrame({lang: [panx_ch[lang][split].num_rows for
                        split in ds], index=['Número de observações de treino']})
```

Saída:

	pt	en	fr
Número de observações de treino	2600	14400	3000

Como temos muito mais exemplos da língua inglesa por design, vamos começar por ela. Primeiramente vamos treinar nosso modelo na língua inglesa e testar ele nas línguas inglesa, portuguesa e francesa.

Vamos inspecionar um exemplo no dataset inglês:

```
exemplo = panx_ch['en']['train'][0]
for key, value in exemplo.items():
    print(f'{key}: {value}')
```

Saída:

```
tokens: ['"', "'", 'Toronto', 'Lynx', "'", '"']
ner_tags: [0, 0, 3, 4, 0, 0]
langs: ['en', 'en', 'en', 'en', 'en', 'en']
```

Podemos ver que a coluna `ner_tags` corresponde a um código mapeado. Então, vamos transformá-lo em uma nova coluna mais familiar, com as tags LOC, PER e ORG. Podemos fazer isso através do atributo `features`.

```
for key, value in panx_ch['en']['train'].features.items():
    print(f'{key}: {value}')
```

Saída:

```
tokens: Sequence(feature=Value(dtype='string', id=None),
length=-1, id=None)
ner_tags: Sequence(feature=ClassLabel(names=['O', 'B-PER',
'I-PER', 'B-ORG', 'I-ORG', 'B-LOC', 'I-LOC'], id=None), length=-1, id=None)
langs: Sequence(feature=Value(dtype='string', id=None),
length=-1, id=None)
```

Podemos ver que dentro do atributo `features` temos o `ner_tags`, que possui um objeto `ClassLabel` com os nomes de cada tag mapeados. Agora vamos conceber uma função que crie a coluna usando o que descobrimos.

```
tags = panx_ch['en']['train'].features['ner_tags'].feature
def create_tag_names_column(batch):
    return {'ner_tags_str': [tags.int2str(idx) for idx in
batch['ner_tags']]
panx_en = panx_ch['en'].map(create_tag_names_column)
exemplo = panx_en['train'][0]
pd.DataFrame([exemplo['tokens'], exemplo['ner_tags_str']],
['tokens', 'tags'])
```

Saída:

	0	1	2	3	4	5
tokens	'	'	Toronto	Lynx	'	'
tags	O	O	B-ORG	I-ORG	O	O

Já vemos alguns desafios que enfrentamos lidando com idiomas: Toronto é uma cidade que seria classificada como local, mas Toronto Lynx é um time de futebol, então é classificado como organização. Outro desafio importante é a criação das tags: podemos ver que é bem trabalhoso e complexo taggear todas as sentenças de um dataset inteiro para treinarmos o modelo.

Dito isso, vamos entender a distribuição de tags no nosso dataset.

```
from collections import Counter
split2freqs = defaultdict(Counter)
for split, dataset in panx_en.items():
    for row in dataset['ner_tags_str']:
        for tag in row:
            if tag.startswith('B'):
                tag_type = tag.split('-')[1]
                split2freqs[split][tag_type] += 1
pd.DataFrame.from_dict(split2freqs, orient='index')
```

Saída:

	ORG	PER	LOC
train	6715	6542	6767
validation	3364	3276	3458
test	3385	3339	3357

As distribuições estão bem homogêneas entre os datasets, o que é ótimo.

Agora, vamos para o modelo. Nesse exemplo, utilizaremos o [XLM-RoBERTa](#). O modelo RoBERTa modifica o pré-treinamento do BERT, melhorando o modelo ao treinar em mais dados por mais tempo, além de não realizar a tarefa de Next Sentence Prediction.

O XLM-RoBERTa, por sua vez, estende o RoBERTa para um pré-treinamento multilíngua. Outra diferença para o BERT é a tokenização: enquanto o BERT tokeniza usando WordPiece, o XLM-RoBERTa utiliza o SentencePiece. Essa mudança de tokenização é especialmente importante para lidar com línguas em que a separação por espaço não faz sentido, como na língua japonesa, sendo mais agnóstica.

```
from transformers import XLMRobertaForTokenClassification,
AutoTokenizer, AutoConfig
import torch
model_name = 'xlm-roberta-base'
tokenizer = AutoTokenizer.from_pretrained(model_name)
index2tag = {idx: tag for idx, tag in enumerate(tags.names)}
tag2index = {tag: idx for idx, tag in enumerate(tags.names)}
config = AutoConfig.from_pretrained(
    model_name,
    num_labels=tags.num_classes,
    id2label=index2tag,
    label2id=tag2index
)
device = torch.device('cuda' if torch.cuda.is_available()
else 'cpu')
model = XLMRobertaForTokenClassification.from_pretrained(model_name,
config=config).to(device)
```

Antes de realizar o fine-tuning do modelo, primeiro precisamos tokenizar nosso dataset e alinhar e limpar nossos labels.

Não queremos que tokens de carácter especial, como o <s> e o <\s> que indicam início e fim de sentença, sejam classificados. Também queremos ignorar a classificação das subpalavras: somente a primeira deve ser classificada, as demais serão tratadas posteriormente quando juntarmos os tokens em palavras. Para isso, vamos usar os word_ids. Vamos ver um exemplo a seguir.

```
tokenized_input = tokenizer(exemplo['tokens'],
is_split_into_words=True)
```

```
tokens =
tokenizer.convert_ids_to_tokens(tokenized_input["input_ids"])
word_ids = tokenized_input.word_ids()
pd.DataFrame([tokens, word_ids], index=["Tokens", "Word
Ids"])
```

Saída:

	0	1	2	3	4	5	6	7	8
Tokens	<s>	_'	_''	_Toronto	_Lyn	x	_''	_'	</s>
Word Ids	None	0	1	2	3	3	4	5	None

Podemos ver que os caracteres especiais estão com Ids None. Além disso, as subpalavras possuem o mesmo id da anterior: a palavra Lynx foi dividida em _Lyn e x, por exemplo, ambas com o id 3. Com isso, podemos tratar os labels para quando forem None ou iguais ao anterior e alocamos o label id -100. Porque -100? Porque a classe `torch.nn.CrossEntropyLoss` possui um atributo chamado `ignore_index` com valor -100. Dessa forma, os tokens associados com label -100 serão ignorados durante o treinamento. Vamos para nosso tratamento?

```
def tokenize_and_align_labels(examples):
    tokenized_inputs = tokenizer(examples['tokens'],
truncation=True, is_split_into_words=True)
    labels = []
    for idx, label in enumerate(examples['ner_tags']):
        word_ids = tokenized_inputs.word_ids(batch_index=idx)
        previous_word_idx = None
        labels_ids = []
        for word_idx in word_ids:
            if word_idx is None or word_idx ==
previous_word_idx:
                labels_ids.append(-100)
            else:
                labels_ids.append(label[word_idx])
                previous_word_idx = word_idx
        labels.append(labels_ids)
    tokenized_inputs['labels'] = labels
    return tokenized_inputs
def encode_pax_dataset(corpus):
    return corpus.map(tokenize_and_align_labels, batched=True,
remove_columns=['langs', 'ner_tags', 'tokens'])
pax_en_encoded = encode_pax_dataset(pax_ch['en'])
```

A avaliação de um modelo NER é similar ao de modelos de classificação, usando métricas como acuracidade, precision, recall e f1-score. A diferença é que todas as palavras de uma entidade precisam ser previstas corretamente para uma previsão contar como correta. Para isso, usaremos a biblioteca [segeval](#), que faz exatamente isso. Veja um exemplo a seguir.

```
from segeval.metrics import classification_report
y_true = [['O', 'O', 'O', 'B-MISC', 'I-MISC', 'I-MISC',
'O'], ['B-PER', 'I-PER', 'O']]
y_pred = [['O', 'O', 'B-MISC', 'I-MISC', 'I-MISC', 'I-
MISC', 'O'], ['B-PER', 'I-PER', 'O']]
classification_report(y_true, y_pred)
```

Saída:

	precision	recall	f1-score	support
MISC	0.00	0.00	0.00	1
PER	1.00	1.00	1.00	1
micro avg	0.50	0.50	0.50	2
macro avg	0.50	0.50	0.50	2
weighted avg	0.50	0.50	0.50	2

Para usar essa biblioteca, precisaremos formatar as previsões e os labels para o formato de lista de listas, conforme visto anteriormente, em que cada lista corresponde a um exemplo do dataset.

```
import numpy as np
def align_predictions(predictions, label_ids):
    preds = np.argmax(predictions, axis=2)
    batch_size, seq_len = preds.shape
    labels_list, preds_list = [], []
    for batch_idx in range(batch_size):
        example_label, example_preds = [], []
        for seq_idx in range(seq_len):
            if label_ids[batch_idx, seq_idx] != -100:
                example_label.append(
                    index2tag[label_ids[batch_idx][seq_idx]]
                )
            example_preds.append(
                index2tag[preds[batch_idx][seq_idx]]
            )
        labels_list.append(example_label)
        preds_list.append(example_preds)
    return preds_list, labels_list
```

Agora temos tudo que precisamos para realizar o fine-tuning do XLM-RoBERTa. Primeiramente, faremos somente no subset do idioma inglês e testaremos o modelo nos demais idiomas, português e francês. É interessante notar que essa parte já fica bem parecida com a da aula passada de análise de sentimento.

```
from transformers import TrainingArguments, Trainer
num_epochs = 3
batch_size = 24
logging_steps = len(panx_en_encoded['train']) // batch_size
finetuned_model_name = f'{model_name}-finetuned-panx-en'
training_args = TrainingArguments(
    output_dir=finetuned_model_name,
    log_level='error',
    num_train_epochs=num_epochs,
    per_device_train_batch_size=batch_size,
    per_device_eval_batch_size=batch_size,
    evaluation_strategy='epoch',
    save_steps=1e6,
    weight_decay=0.01,
    disable_tqdm=False,
    logging_steps=logging_steps,
    push_to_hub=False
)
from seqeval.metrics import f1_score
def compute_metrics(eval_pred):
    y_pred, y_true = align_predictions(
        eval_pred.predictions, eval_pred.label_ids
    )
    return {'f1': f1_score(y_true, y_pred)}
```

Por fim, precisamos aplicar o método pad para deixar todas as sequências do mesmo tamanho em um batch (lote). Para isso, usaremos o data collator. Precisamos aplicá-lo tanto no texto como nos labels, porque agora temos labels de tamanhos diferentes.

São labels sequenciais, ao contrário do que tínhamos na aula passada na análise de sentimento, em que havia somente a previsão de uma classe. O pad seguirá a mesma lógica implementada, transformando os tokens pad em ids de -100 para serem ignorados durante o treinamento.

```
from transformers import DataCollatorForTokenClassification
data_collator = DataCollatorForTokenClassification(tokenizer)
```



```

        trainer = Trainer(model, args=training_args,
data_collator=data_collator, compute_metrics=compute_metrics,
train_dataset=panx_en_encoded['train'],
eval_dataset=panx_en_encoded['validation'],
tokenizer=tokenizer)
        trainer.train()

```

Saída:

```

        {'loss': 0.1034, 'grad_norm': 10.835453987121582,
'learning_rate': 3.3333333333333335e-05, 'epoch': 1.0}
        {'eval_loss': 0.3764185905456543, 'eval_f1':
0.7797924949093377, 'eval_runtime': 393.4814,
'eval_samples_per_second': 18.298, 'eval_steps_per_second':
0.762, 'epoch': 1.0}
        {'loss': 0.1558, 'grad_norm': 5.884861469268799,
'learning_rate': 1.6666666666666667e-05, 'epoch': 2.0}
        {'eval_loss': 0.3161863684654236, 'eval_f1':
0.8053080011752032, 'eval_runtime': 392.6699,
'eval_samples_per_second': 18.336, 'eval_steps_per_second':
0.764, 'epoch': 2.0}
        {'loss': 0.0934, 'grad_norm': 2.8271751403808594,
'learning_rate': 0.0, 'epoch': 3.0}
        {'eval_loss': 0.32673001289367676, 'eval_f1':
0.8179060665362037, 'eval_runtime': 354.2475,
'eval_samples_per_second': 20.325, 'eval_steps_per_second':
0.847, 'epoch': 3.0}
        {'train_runtime': 14352.5169, 'train_samples_per_second':
3.01, 'train_steps_per_second': 0.125, 'train_loss':
0.11755364523993599, 'epoch': 3.0}
        TrainOutput(global_step=1800,
training_loss=0.11755364523993599, metrics={'train_runtime':
14352.5169, 'train_samples_per_second': 3.01,
'train_steps_per_second': 0.125, 'total_flos':
840013140205008.0, 'train_loss': 0.11755364523993599, 'epoch':
3.0})

```

Treinar o modelo pode demorar bastante, tenha paciência. Mas podemos ver que os resultados foram muito bons, com F1 de cerca de 82%! Após isso já teremos nosso modelo treinado, agora é só formatar o resultado final:

```
def tag_text(text, tags, model, tokenizer):
    tokens = tokenizer(text).tokens()
    input_ids = tokenizer(text,
return_tensors='pt').input_ids.to(device)
    outputs = model(input_ids)[0]
    predictions = torch.argmax(outputs, dim=2)
    preds = [tags.names[p] for p in
predictions[0].cpu().numpy()]
    return pd.DataFrame([tokens, preds], index=['Tokens',
'Tags'])
```

Nosso modelo, até então, foi treinado somente com texto do idioma inglês. Será que ele já performa bem nos idiomas português e francês?

```
def get_f1_score(trainer, dataset):
    return trainer.predict(dataset).metrics['test_f1']

def evaluate_lang_performance(lang, trainer):
    panx_ds = encode_panx_dataset(panx_ch[lang])
    return get_f1_score(trainer, panx_ds['test'])
f1_scores = defaultdict(dict)
for lang in langs:
    f1_scores['en'][lang] = evaluate_lang_performance(lang,
trainer)
f1_scores
```

Saída:

```
defaultdict(<class 'dict'>, {'en': {'pt':
0.7679671457905545, 'en': 0.8178026399725207, 'fr':
0.7699484409526148}})
```

Incrível! Mesmo o treinamento sendo no idioma inglês, já conseguimos usar esse modelo nos outros idiomas com uma boa performance, com F1 de cerca de 77%.

Mas ainda é possível melhorar: podemos treinar um modelo em todos os idiomas de uma vez.

```
from datasets import concatenate_datasets
def concatenate_splits(corpus):
```

```

multi_corpus = DatasetDict()
for split in corpus[0].keys():
    multi_corpus[split] = concatenate_datasets(
        [data[split] for data in corpus]
    ).shuffle(seed=42)
return multi_corpus
panx_all_encoded = [panx_en_encoded]
for lang in ['pt', 'fr']:
    panx_encoded = encode_pnx_dataset(panx_ch[lang])
    panx_all_encoded.append(panx_encoded)
corpus = concatenate_splits(panx_all_encoded)
training_args.output_dir = f'{model_name}-finetuned-panx-
all'
training_args.logging_steps = len(corpus['train']) //
batch_size
trainer = Trainer(model, args=training_args,
data_collator=data_collator, compute_metrics=compute_metrics,
train_dataset=corpus['train'],
eval_dataset=corpus['validation'], tokenizer=tokenizer)
trainer.train()

```

Saída:

```

{'loss': 0.1942, 'grad_norm': 11.842309951782227,
'learning_rate': 3.3353317346123105e-05, 'epoch': 1.0}
{'eval_loss': 0.29228514432907104, 'eval_f1':
0.8157923201730666, 'eval_runtime': 305.4299,
'eval_samples_per_second': 32.741, 'eval_steps_per_second':
1.365, 'epoch': 1.0}
{'loss': 0.1225, 'grad_norm': 10.712843894958496,
'learning_rate': 1.6706634692246204e-05, 'epoch': 2.0}
{'eval_loss': 0.31587448716163635, 'eval_f1':
0.8195355240752147, 'eval_runtime': 2602.45,
'eval_samples_per_second': 3.843, 'eval_steps_per_second':
0.16, 'epoch': 2.0}
{'eval_loss': 0.3558487892150879, 'eval_f1':
0.8312026943830805, 'eval_runtime': 323.4996,
'eval_samples_per_second': 30.912, 'eval_steps_per_second':
1.289, 'epoch': 3.0}
{'train_runtime': 95680.6525, 'train_samples_per_second':
0.627, 'train_steps_per_second': 0.026, 'train_loss':
0.12965314505959777, 'epoch': 3.0}
TrainOutput(global_step=2502,
training_loss=0.12965314505959777, metrics={'train_runtime':
95680.6525, 'train_samples_per_second': 0.627,
'train_steps_per_second': 0.026, 'total_flos':
1143126996806784.0, 'train_loss': 0.12965314505959777, 'epoch':
3.0})

```

Que resultado! 83% de F1 score no total! Agora vamos testar o modelo multilinguagem nos nossos datasets e ver como ficou.

```
for idx, lang in enumerate(langs):
    f1_scores['all'][lang] = get_f1_score(
        trainer, panx_all_encoded[idx]['test']
    )
scores_data = {
    'en': f1_scores['en'],
    'all': f1_scores['all'],
}
f1_scores_df =
pd.DataFrame(scores_data).T.round(4).rename_axis(
    index='Fine-tuned on', columns='Evaluated on'
)
f1_scores_df
```

Saída:

Evaluated on	pt	en	fr
Fine-tuned on			
en	0.7680	0.8178	0.7699
all	0.8216	0.8727	0.8511

Veja que usar outros idiomas no treinamento aumentou o resultado testando somente no dataset em inglês também, indo de 82% para 87% em F1! Isso porque ele aprendeu outros padrões com os outros idiomas que puderam ser aproveitados. No entanto, isso é verdade apenas para aqueles idiomas que são similares, conforme comentamos anteriormente: todos os idiomas testados pertencem à família Indo-Europeia, que compartilham padrões entre si.

Esses padrões fazem com que o modelo performe de maneira quase equivalente nesses idiomas. Podemos ver que o F1 em todos os três idiomas ficou bem parecido, em torno de 85%, o que é muito bom para um modelo de Reconhecimento de Entidade Nomeada.

O QUE VOCÊ VIU NESTA AULA?

Nessa aula você aprendeu como desenvolver e usar modelos para o reconhecimento de entidades nomeadas, tanto para um idioma específico como multi-idioma. Agora você já pode extrair informações preciosas dos dados para análises, extração de variáveis para outros modelos e muito mais.



REFERÊNCIAS

GITHUB. **sequeval**. 2024. Disponível em: <<https://github.com/chakki-works/sequeval>>. Acesso em: 25 jul. 2024.

HUGGING FACE. **monilouise/ner_news_portuguese**. 2024. Disponível em: <https://huggingface.co/monilouise/ner_news_portuguese>. Acesso em: 25 jul. 2024.

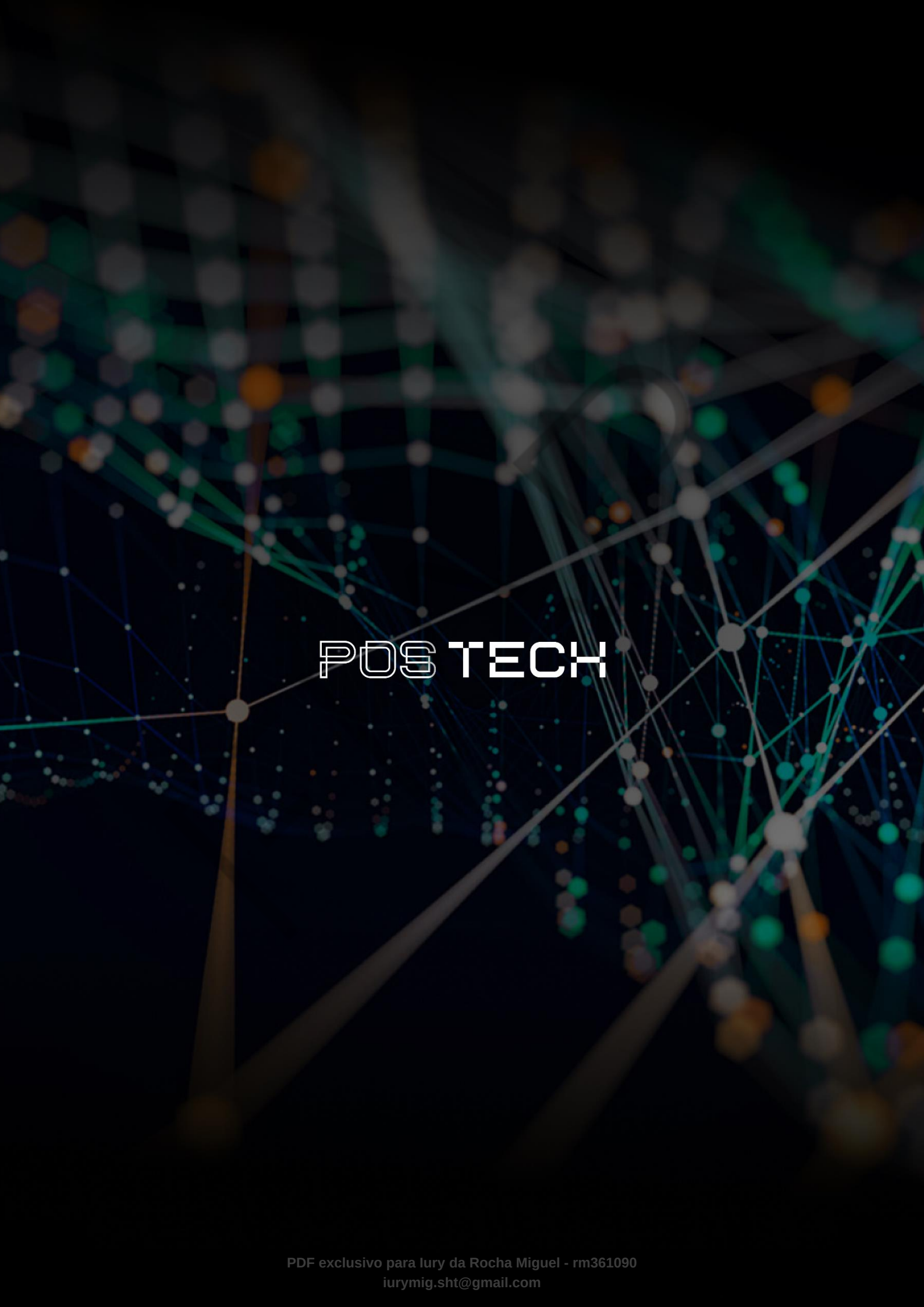
HUGGING FACE. **XLM-RoBERTa**. 2024. Disponível em: <https://huggingface.co/docs/transformers/model_doc/xlm-roberta>. Acesso em: 25 jul. 2024.

TUNSTALL, L.; VON WERRA, L.; WOLF, T. **Natural Language Processing with Transformers**. Revised Edition. [s.l.]: O'Reilly Media, Inc., 2022.

PALAVRAS-CHAVE

Processamento de Linguagem Natural. Transformers. Transferência de Conhecimento. Bert. Aprendizado de Máquina. Extração de Entidades Nomeadas. Reconhecimento de Entidades Nomeadas.

EMENDAS

The background is a dark, abstract network visualization. It features a complex web of thin, light-colored lines connecting numerous small, glowing nodes. The nodes are primarily white and light blue, with some larger, more prominent nodes in green and orange. The overall effect is a sense of dynamic connectivity and data flow, typical of a digital or biological network.

POSTECH